

**CUSTOMER SEGMENTATION USING RFM  
ANALYSIS**

**IE6400 FOUNDATIONS DATA ANALYTICS  
ENGINEERING**

**FALL 2024**

**Project Report**

**Group Number 9:**

Swathi Baba Eswarappa

Himabindu peramala

Rohan Prakash Krishna Prakash

## Introduction

Understanding consumer behavior is essential for developing successful marketing strategies and improving company outcomes in the dynamic world of e-commerce. In order to understand the intricacies of consumer behavior inside an e-commerce dataset, this research explores the use of exploratory data analysis (EDA) and sophisticated segmentation approaches. Using clustering techniques and RFM analysis (Recency, Frequency, Monetary), the study seeks to provide practical insights that will support data-driven decision-making.

Purchase preferences, engagement levels, and transactional patterns all have an impact on the complex phenomena of customer behavior. This research aims to uncover hidden trends, pinpoint high-value client categories, and provide customized marketing suggestions by closely analyzing the information. Through statistical calculation, methodical preprocessing, and visualization, the study offers a structured framework for better understanding consumer dynamics.

The research combines subject expertise with sophisticated analytics to divide clients into discrete groups according to their buying patterns using technologies like Python and its extensive data libraries. Businesses may use this information as a basis to improve customer engagement, retention, and profitability.

The ultimate goal of this research is to provide stakeholders a sophisticated grasp of e-commerce consumer segmentation in order to promote evidence-based tactics that enhance customer connections and promote long-term success. This investigation contributes to the continuous effort to develop more individualized and successful marketing strategies in the digital era by delving into the complexities of consumer data.

## Data Preprocessing and Analysis

### 1. Data Preprocessing:

In the data Preprocessing phase, we utilized the Python programming language and its data manipulation library, pandas, to access and load the dataset for our analysis. The dataset, titled 'ecommercedata.csv', was obtained from the file path specified as C:\project2\IE6400. This CSV file contains e-commerce-related data intended for analysis.

By employing the pandas library, we imported and structured the dataset in a way that facilitates its manipulation and analysis. The `pd.read_csv` function was used to read the contents of the CSV file and create a DataFrame, a tabular data structure commonly used in data analysis. This DataFrame, named `ed`, serves as the foundation for our subsequent preprocessing and exploratory data analysis.

```

● import pandas as pd
  ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
  print(ed)

```

## Displaying the First Few Rows:

The first and most straightforward step is to display the initial rows of the dataset. This allows us to visually inspect the data and gain insights into its format and contents. The Pandas library provides a convenient function called `head()` to facilitate this. Here's an example of how it's used

```

#Displaying & Reading the first 10 rows of the dataset
ed.head(10)

```

### Result:

|   | InvoiceNo | StockCode | Description                         | Quantity | InvoiceDate      | UnitPrice | CustomerID | Country        |
|---|-----------|-----------|-------------------------------------|----------|------------------|-----------|------------|----------------|
| 0 | 536365    | 85123A    | WHITE HANGING HEART T-LIGHT HOLDER  | 6        | 12-01-2010 08:26 | 2.55      | 17850.0    | United Kingdom |
| 1 | 536365    | 71053     | WHITE METAL LANTERN                 | 6        | 12-01-2010 08:26 | 3.39      | 17850.0    | United Kingdom |
| 2 | 536365    | 84406B    | CREAM CUPID HEARTS COAT HANGER      | 8        | 12-01-2010 08:26 | 2.75      | 17850.0    | United Kingdom |
| 3 | 536365    | 84029G    | KNITTED UNION FLAG HOT WATER BOTTLE | 6        | 12-01-2010 08:26 | 3.39      | 17850.0    | United Kingdom |
| 4 | 536365    | 84029E    | RED WOOLLY HOTTIE WHITE HEART.      | 6        | 12-01-2010 08:26 | 3.39      | 17850.0    | United Kingdom |
| 5 | 536365    | 22752     | SET 7 BABUSHKA NESTING BOXES        | 2        | 12-01-2010 08:26 | 7.65      | 17850.0    | United Kingdom |
| 6 | 536365    | 21730     | GLASS STAR FROSTED T-LIGHT HOLDER   | 6        | 12-01-2010 08:26 | 4.25      | 17850.0    | United Kingdom |
| 7 | 536366    | 22633     | HAND WARMER UNION JACK              | 6        | 12-01-2010 08:28 | 1.85      | 17850.0    | United Kingdom |
| 8 | 536366    | 22632     | HAND WARMER RED POLKA DOT           | 6        | 12-01-2010 08:28 | 1.85      | 17850.0    | United Kingdom |
| 9 | 536367    | 84879     | ASSORTED COLOUR BIRD ORNAMENT       | 32       | 12-01-2010 08:34 | 1.69      | 13047.0    | United Kingdom |

## Data Cleaning:

We have methodically found and measured missing data in our dataset throughout the step of recognizing and managing null values. The code snippet '`ed.isnull().sum()`' run on the DataFrame 'ed' checks for null (or missing) values across all columns. Cells in the dataset with invalid values are identified using the `isnull()` method, which marks them as "True" otherwise and "False" otherwise. These 'True' values are then totaled by the `sum()` method that follows, therefore tallying the amount of null entries for every column. We learned a lot about the distribution and amount of missing data in our dataset by running this code. These results will help us decide how best to fill in these gaps throughout the data preparation and cleaning stage.

```
#Identifying missing values in the dataset
ed.isnull().sum()
```

### Result:

```
InvoiceNo      0
StockCode      0
Description    1454
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64
```

## Handling Null Values:

'Description' and CustomerID' were among the variables we found to have missing values. These were replaced with the proper placeholders to denote the absence or ambiguous information. We improved data quality, simplified the dataset, and concentrated on more insightful factors for additional research by removing these columns.

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
#Handling missing values in the dataset
ed['Description'].fillna('0', inplace=True)
ed['CustomerID'].fillna('0', inplace=True)
missing_data = ed.isnull().sum()
print(missing_data)
```

### Result:

```
InvoiceNo      0
StockCode      0
Description     0
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID     0
Country        0
dtype: int64
```

## Data Type Conversion for Date and Time Fields:

**Converting Date Fields:** The dataset's "InvoiceDate" field correspond to the dates of occurrence and reported events, respectively. We converted these attributes from object data types to date data types using "pd.to\_datetime()" in order to guarantee data accuracy and allow for chronological analysis. This conversion ensures accurate date recognition and processing, enabling insightful temporal analysis.

**Converting Time Fields:** The "InvoiceDate" field in the dataset represents the time of customer bought the product. To standardize the time format and support precise time-related analysis, we performed a process to convert the time values. We transformed the "InvoiceDate" field into a consistent time format ("HH:MM: SS") by padding and reformatting the values. This transformation ensures uniform time values suitable for time-based analyses.

```
#Converting InvoiceDate to datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
ed['Date'] = ed['InvoiceDate'].dt.date
ed['Time'] = ed['InvoiceDate'].dt.time
ed[['Date', 'Time']]
```

**Result:**

|        | Date       | Time     |
|--------|------------|----------|
| 0      | 2010-12-01 | 08:26:00 |
| 1      | 2010-12-01 | 08:26:00 |
| 2      | 2010-12-01 | 08:26:00 |
| 3      | 2010-12-01 | 08:26:00 |
| 4      | 2010-12-01 | 08:26:00 |
| ...    | ...        | ...      |
| 541904 | 2011-12-09 | 12:50:00 |
| 541905 | 2011-12-09 | 12:50:00 |
| 541906 | 2011-12-09 | 12:50:00 |
| 541907 | 2011-12-09 | 12:50:00 |
| 541908 | 2011-12-09 | 12:50:00 |

541909 rows × 2 columns

## 2. RFM Calculation:

In marketing and customer relationship management, RFM (Recency, Frequency, Monetary) analysis is a potent consumer segmentation method. Three factors are used to assess client value: monetary (the total amount spent by a customer), frequency (the frequency of a customer's purchases), and recency (the recentness of a customer's purchases). RFM assists companies in locating devoted, valuable clients, customizing marketing plans, and improving customer retention initiatives.

```

import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
# Convert 'InvoiceDate' to datetime if not done yet
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
# Calculate the total purchase amount for each transaction
ed['Amount'] = ed['Quantity'] * ed['UnitPrice']
# Set a reference date (typically the latest date in the data) to calculate Recency
reference_date = ed['InvoiceDate'].max()
# Initialize an empty RFM DataFrame and set CustomerID as the index
rfm = pd.DataFrame(index=ed['CustomerID'].unique())

# Calculate Recency
# For each customer, calculate days since last purchase
rfm['Recency'] = ed.groupby('CustomerID')['InvoiceDate'].max().apply(lambda x: (reference_date - x).days)

# Calculate Frequency
# Count unique invoices per customer
rfm['Frequency'] = ed.groupby('CustomerID')['InvoiceNo'].nunique()

# Calculate Monetary
# Sum the total spending for each customer
rfm['Monetary'] = ed.groupby('CustomerID')['Amount'].sum()

# Display the resulting RFM DataFrame
rfm.reset_index(inplace=True) # Optional: Reset index to include CustomerID as a column
rfm.head(10)

```

## Result:

|   | index   | Recency | Frequency | Monetary |
|---|---------|---------|-----------|----------|
| 0 | 17850.0 | 301.0   | 35.0      | 5288.63  |
| 1 | 13047.0 | 31.0    | 18.0      | 3079.10  |
| 2 | 12583.0 | 2.0     | 18.0      | 7187.34  |
| 3 | 13748.0 | 95.0    | 5.0       | 948.25   |
| 4 | 15100.0 | 333.0   | 6.0       | 635.10   |
| 5 | 15291.0 | 31.0    | 20.0      | 4596.51  |
| 6 | 14688.0 | 7.0     | 27.0      | 5107.38  |
| 7 | 17809.0 | 60.0    | 15.0      | 4627.62  |
| 8 | 15311.0 | 0.0     | 118.0     | 59419.34 |
| 9 | 14527.0 | 2.0     | 86.0      | 7711.38  |

## Recency:

Recency gives information about a customer's degree of involvement by counting the days since their last transaction. The *InvoiceDate* column is used to extract the purchase dates from the dataset and convert them to a datetime format for precise processing in order to compute recency. Each customer's most recent purchase date is determined using the `max()` method, which groups them according to their distinct *CustomerID*. The difference in days between the customer's previous transaction and a reference date usually the most recent date in the dataset is calculated. This procedure shows how recently each client has dealt with the company.

```
import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
# Ensure InvoiceDate is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
# Set the reference date (latest date in the dataset)
reference_date = ed['InvoiceDate'].max()
# Calculate the last purchase date for each customer
recency_df = ed.groupby('CustomerID').agg(LastPurchase=('InvoiceDate', 'max')).reset_index()
# Calculate recency as the number of days since the last purchase
recency_df['Recency'] = (reference_date - recency_df['LastPurchase']).dt.days
# Display the results
print("Recency Table:")
print(recency_df.head())
```

## Result:

```
Recency Table:
   CustomerID  LastPurchase  Recency
0      12346.0           NaT       NaN
1      12347.0 2011-12-07 15:52:00     1.0
2      12348.0 2011-04-05 10:47:00   248.0
3      12349.0           NaT       NaN
4      12350.0 2011-02-02 16:01:00   309.0
```

## Frequency(F):

One of the most important metrics in RFM analysis is frequency, which quantifies how frequently a client makes purchases over a certain time frame. It shows how involved and consistent a consumer is with their purchases. The `nunique()` method is used to count the number of unique invoices (*InvoiceNo*) connected to each customer after the dataset has been aggregated by the *CustomerID* column in order to determine frequency. This gives each customer's total number of unique transactions. The resultant values are changed to 'Frequency' for clarity, which helps organizations select their most frequent customers for marketing or loyalty programs.

```
import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
# Calculate Frequency: Count the unique 'InvoiceNo' for each customer
frequency_df = ed.groupby('CustomerID').agg({
    'InvoiceNo': 'nunique'
}).reset_index()

# Rename 'InvoiceNo' to 'Frequency'
frequency_df.rename(columns={'InvoiceNo': 'Frequency'}, inplace=True)
# Display the Frequency values
frequency_df.head(10)
```

## Result:

|   | CustomerID | Frequency |
|---|------------|-----------|
| 0 | 12346.0    | 2         |
| 1 | 12347.0    | 7         |
| 2 | 12348.0    | 4         |
| 3 | 12349.0    | 1         |
| 4 | 12350.0    | 1         |
| 5 | 12352.0    | 11        |
| 6 | 12353.0    | 1         |
| 7 | 12354.0    | 1         |
| 8 | 12355.0    | 1         |
| 9 | 12356.0    | 3         |

## Monetary(M):

One important statistic in RFM analysis is monetary, which measures the total amount of money a client has spent with a company over a given time frame. It assists in determining the worth of a consumer by looking at their purchasing patterns. *Quantity* and *UnitPrice* are multiplied to determine the entire purchase amount for each transaction, which is then recorded in a new column called *Amount*. After grouping the dataset by *CustomerID*, a pivot table with the aggregation function *sum* is used to get the total amount for each client. 'Monetary' is the new label given to the generated figures to properly show overall spending.

```
import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
#Calculate the total purchase amount for each transaction
ed['Amount'] = ed['Quantity'] * ed['UnitPrice']
# Calculate Monetary: Sum the 'Amount' for each customer using pivot_table
monetary_df = ed.pivot_table(
    index='CustomerID',
    values='Amount',
    aggfunc='sum'
).reset_index()
# Rename the 'Amount' column to 'Monetary'
monetary_df.rename(columns={'Amount': 'Monetary'}, inplace=True)
# Display the Monetary values
monetary_df.head(10)
```

## Result:

|   | CustomerID | Monetary |
|---|------------|----------|
| 0 | 12346.0    | 0.00     |
| 1 | 12347.0    | 4310.00  |
| 2 | 12348.0    | 1797.24  |
| 3 | 12349.0    | 1757.55  |
| 4 | 12350.0    | 334.40   |
| 5 | 12352.0    | 1545.41  |
| 6 | 12353.0    | 89.00    |
| 7 | 12354.0    | 1079.40  |
| 8 | 12355.0    | 459.40   |
| 9 | 12356.0    | 2811.43  |

## 3. RFM Segmentation:

RFM (Recency, Frequency, Monetary) segmentation is a data analysis method that helps firms analyze consumer behavior and customize marketing tactics by classifying clients according to their transaction history. The three main variables it looks at are Recency, which indicates how recently a client made a purchase, Frequency, which indicates how frequently they make purchases, and Monetary, which indicates how much money they spend. Businesses may identify high-value consumers, identify at-risk customers, and customise offers for certain customer categories by using these three characteristics to score customers. Businesses may enhance marketing campaigns, maximize client retention, and boost profitability via RFM segmentation.

Recency, Frequency, and Monetary (RFM) segmentation of e-commerce transaction data is carried out in the study using the pandas package. The *InvoiceDate* column is converted to a datetime format by `to_datetime()` once the data has been loaded using the `read_csv()` method. *Quantity* and *UnitPrice* are multiplied to determine the total transaction value. `Dropna()` is used to manage missing data. The RFM metrics for every client are determined via the `groupby()` and aggregation procedures like `nunique()` and `sum()`. Recency, frequency, and monetary values are categorized using `pd.cut()` to apply custom scoring, which is then combined into a final RFM score for segmentation.

```

import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
# Convert 'InvoiceDate' to datetime
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
# Calculate the total purchase amount for each transaction
ed['Amount'] = ed['Quantity'] * ed['UnitPrice']
# Calculate the total price for each transaction
ed['Amount'] = ed['Quantity'] * ed['UnitPrice']
# Drop rows with null values
ed.dropna(inplace=True)
# Set a reference date (typically the latest date in the dataset)
reference_date = ed['InvoiceDate'].max()
# Calculate Recency, Frequency, and Monetary metrics for each customer
rfm = ed.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (reference_date - x.max()).days, # Recency
    'InvoiceNo': 'nunique', # Frequency
    'Amount': 'sum' # Monetary
}).reset_index()
# Rename columns
rfm.rename(columns={
    'InvoiceDate': 'Recency',
    'InvoiceNo': 'Frequency',
    'Amount': 'Monetary'
}, inplace=True)
# Define custom bins and labels
# Recency Score: Lower recency gets a higher score
recency_bins = [0, 30, 90, 180, rfm['Recency'].max()]
rfm['R_Score'] = pd.cut(rfm['Recency'], bins=recency_bins, labels=[4, 3, 2, 1], right=False)
# Frequency Score: Higher frequency gets a higher score
frequency_bins = [0, 2, 5, 10, rfm['Frequency'].max()]
rfm['F_Score'] = pd.cut(rfm['Frequency'], bins=frequency_bins, labels=[1, 2, 3, 4], right=False)
# Monetary Score: Higher monetary value gets a higher score
monetary_bins = [0, 100, 500, 1000, rfm['Monetary'].max()]
rfm['M_Score'] = pd.cut(rfm['Monetary'], bins=monetary_bins, labels=[1, 2, 3, 4], right=False)
# Combine the RFM scores into a single 'RFM_Score' column
rfm['RFM_Score'] = rfm['R_Score'].astype(str) + rfm['F_Score'].astype(str) + rfm['M_Score'].astype(str)

# Print interpretation for RFM scores (1 to 4) for Recency, Frequency, and Monetary
print("\nInterpretation of RFM scores:")
print("Recency (R) Score:")
print("4: Customer made a very recent purchase")
print("3: Customer purchased fairly recently")
print("2: Customer's last purchase was somewhat recent")
print("1: Customer has not made a purchase in a long time\n")

print("Frequency (F) Score:")
print("4: Customer has made many purchases")
print("3: Customer has made a moderate number of purchases")
print("2: Customer has made a few purchases")
print("1: Customer has made very few purchases\n")

print("Monetary (M) Score:")
print("4: Customer has spent a high amount overall")
print("3: Customer has spent a moderately high amount")
print("2: Customer has spent a moderate amount")
print("1: Customer has spent a low amount overall")

# Display the cleaned data and RFM scores
print("Cleaned data with RFM metrics:")
print(rfm.head(10))

```

## Result:

Interpretation of RFM scores:

Recency (R) Score:

- 4: Customer made a very recent purchase
- 3: Customer purchased fairly recently
- 2: Customer's last purchase was somewhat recent
- 1: Customer has not made a purchase in a long time

Frequency (F) Score:

- 4: Customer has made many purchases
- 3: Customer has made a moderate number of purchases
- 2: Customer has made a few purchases
- 1: Customer has made very few purchases

Monetary (M) Score:

- 4: Customer has spent a high amount overall
- 3: Customer has spent a moderately high amount
- 2: Customer has spent a moderate amount
- 1: Customer has spent a low amount overall

Cleaned data with RFM metrics:

|   | CustomerID | Recency | Frequency | Monetary | R_Score | F_Score | M_Score | RFM_Score |
|---|------------|---------|-----------|----------|---------|---------|---------|-----------|
| 0 | 12347.0    | 1       | 5         | 2540.29  | 4       | 3       | 4       | 434       |
| 1 | 12348.0    | 248     | 1         | 367.00   | 1       | 1       | 2       | 112       |
| 2 | 12350.0    | 309     | 1         | 334.40   | 1       | 1       | 2       | 112       |
| 3 | 12352.0    | 35      | 5         | 456.08   | 3       | 3       | 2       | 332       |
| 4 | 12355.0    | 213     | 1         | 459.40   | 1       | 1       | 2       | 112       |
| 5 | 12356.0    | 245     | 1         | 481.46   | 1       | 1       | 2       | 112       |
| 6 | 12357.0    | 32      | 1         | 6207.67  | 3       | 1       | 4       | 314       |
| 7 | 12358.0    | 1       | 2         | 1168.06  | 4       | 2       | 4       | 424       |
| 8 | 12359.0    | 7       | 4         | 3404.48  | 4       | 2       | 4       | 424       |
| 9 | 12362.0    | 2       | 5         | 2135.79  | 4       | 3       | 4       | 434       |

## 4. Customer Segmentation:

In customer segmentation, K-Means clustering uses variables like Recency, Frequency, and Monetary (RFM) to categorize consumers according to their purchase behavior. Businesses may improve consumer targeting, customize marketing tactics, and make better decisions based on customer group profiles by using the algorithm to group customers into clusters with comparable attributes.

Groupby() and aggregation methods like lambda, nunique(), and sum() are used to compute RFM metrics. To guarantee that every measure makes an equivalent contribution to the clustering process, the data is scaled using StandardScaler. The Elbow Method determines the ideal number of clusters by calculating the Within-Cluster Sum of Squares (WCSS) for different cluster numbers, which are then shown using the plot() function in matplotlib. Once the number of clusters (for example, four clusters) has been established, KMeans() from sklearn.cluster is used to apply K-Means clustering, then fit\_predict() is used to assign consumers to clusters.

Lastly, groupby() and agg() are used to compute the mean RFM values and customer count for each cluster, which summarizes the cluster features. A pair plot made with seaborn.pairplot()

is used to visually assess the clusters; each cluster is color-coded for ease of comprehension. By identifying client groups with comparable habits, this segmentation makes it possible to implement more focused marketing campaigns.

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the dataset
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")

# Drop null values
ed.dropna(inplace=True)

# Convert 'InvoiceDate' to datetime and calculate Amount
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
ed['Amount'] = ed['Quantity'] * ed['UnitPrice']

# Set reference date for Recency calculation
reference_date = ed['InvoiceDate'].max()

# Calculate RFM metrics
rfm = ed.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (reference_date - x.max()).days, # Recency
    'InvoiceNo': 'nunique', # Frequency
    'Amount': 'sum' # Monetary
}).reset_index()

# Rename columns
rfm.rename(columns={'InvoiceDate': 'Recency', 'InvoiceNo': 'Frequency', 'Amount': 'Monetary'}, inplace=True)

# Handle missing or invalid values in RFM metrics
rfm = rfm.dropna()
rfm = rfm[(rfm['Recency'] >= 0) & (rfm['Frequency'] > 0) & (rfm['Monetary'] > 0)]

# Scale RFM data for clustering
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])

# Check for NaN or extreme values in scaled data
if np.isnan(rfm_scaled).any():
    print("NaN values found in scaled data. Please check your input dataset.")
    raise ValueError("NaN values found in scaled data.")
if np.isinf(rfm_scaled).any():
    print("Infinity values found in scaled data. Please check your input dataset.")
    raise ValueError("Infinity values found in scaled data.")

# Find the optimal number of clusters using the Elbow Method
wcss = [] # Within-cluster sum of squares
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(rfm_scaled)
    wcss.append(kmeans.inertia_)

# Plot the Elbow Graph
plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker='o', linestyle='-', color='b')
plt.title('Elbow Method for Optimal Number of Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()

# Choose an appropriate number of clusters (e.g., based on the elbow point) and fit K-Means
optimal_clusters = 4 # Example: Select 4 clusters based on the elbow method
kmeans = KMeans(n_clusters=optimal_clusters, random_state=0)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)
```

```

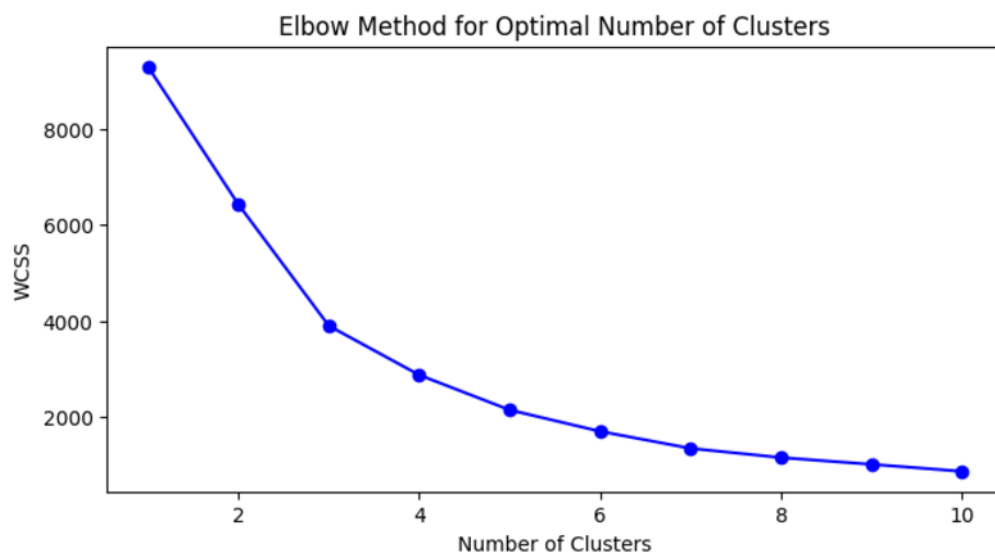
# Analyze and interpret each cluster
# Calculate mean RFM values for each cluster
cluster_summary = rfm.groupby('Cluster').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'CustomerID': 'count'
}).rename(columns={'CustomerID': 'CustomerCount'})

print("Cluster Summary (Mean RFM values and Customer Count):")
print(cluster_summary)

# Visualize clusters using a pair plot
sns.pairplot(rfm, hue='Cluster', palette='viridis', markers=["o", "s", "D", "X"])
plt.show()

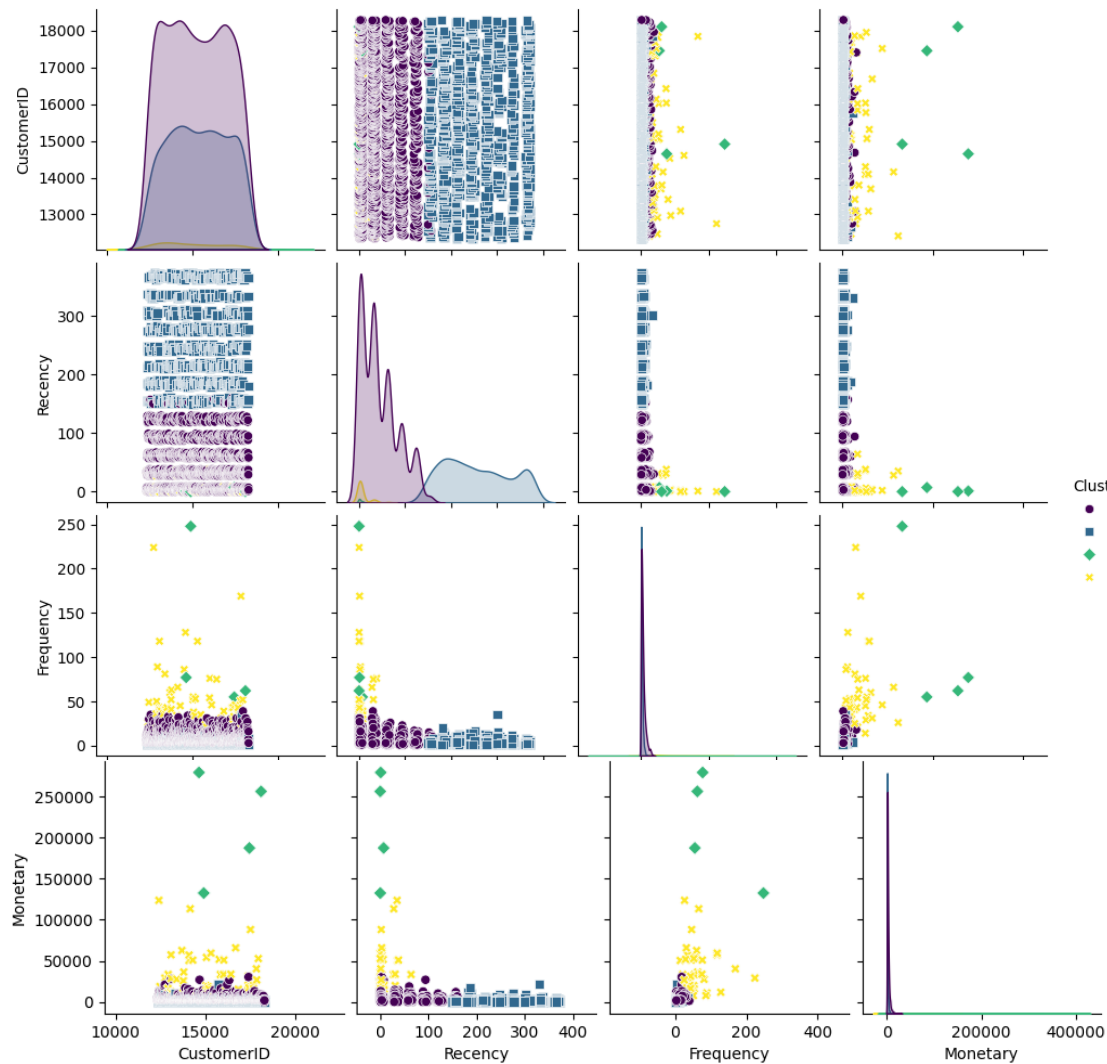
```

## Result:



Cluster Summary (Mean RFM values and Customer Count):

| Cluster | Recency    | Frequency  | Monetary      | CustomerCount |
|---------|------------|------------|---------------|---------------|
| 0       | 43.908040  | 6.656281   | 2080.088825   | 1990          |
| 1       | 255.718363 | 3.286394   | 923.753788    | 1051          |
| 2       | 2.000000   | 110.500000 | 213995.575000 | 4             |
| 3       | 7.269231   | 55.711538  | 32024.793077  | 52            |



## 5. Segment Profiling:

K-Means clustering is used in the segment profiling study to divide up the client base according to Recency, Frequency, and Monetary (RFM) parameters. The RFM metrics are first calculated, where Monetary is the total amount spent by each client, Frequency is the number of unique transactions, and Recency is the number of days since the previous purchase. After grouping by *CustomerID*, these metrics are obtained using aggregation techniques such as `nunique()` and `sum()`.

To guarantee that each measure is given equal importance for clustering, the RFM data is then normalized using `StandardScaler`. After that, K-Means clustering is carried out using four clusters, and each client is allocated to a certain cluster.

In order to better understand the usual features of each client segment, the segment profiling process proceeds by computing the mean RFM values for each cluster. Insights into the relative size of each segment are also provided by the client count for each cluster. Businesses may better understand their clientele and adjust their tactics based on the unique behaviors of each consumer segment thanks to this method.

```

# Calculate RFM metrics
rfm = ed.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (reference_date - x.max()).days, # Recency
    'InvoiceNo': 'nunique', # Frequency
    'Amount': 'sum' # Monetary
}).reset_index()
# Rename columns
rfm.rename(columns={'InvoiceDate': 'Recency', 'InvoiceNo': 'Frequency', 'Amount': 'Monetary'}, inplace=True)
# Handle missing or invalid values in RFM metrics
rfm = rfm.dropna()
rfm = rfm[(rfm['Recency'] >= 0) & (rfm['Frequency'] > 0) & (rfm['Monetary'] > 0)]
# Scale RFM data for clustering
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
# Apply K-Means clustering
kmeans = KMeans(n_clusters=4, random_state=0)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

# Verify that the Cluster column exists
print(rfm.head()) # Ensure the 'Cluster' column is present

# Calculate mean RFM values for each cluster
rfm_summary = rfm.groupby('Cluster').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean'
}).reset_index()

# Calculate customer count for each cluster
customer_counts = rfm.groupby('Cluster').agg({
    'CustomerID': 'count'
}).rename(columns={'CustomerID': 'CustomerCount'}).reset_index()

# Display results
print("Cluster Summary (Mean RFM values):")
print(rfm_summary)
print("\nCustomer Count in Each Cluster:")
print(customer_counts)

```

## Result:

|   | CustomerID | Recency | Frequency | Monetary | Cluster |
|---|------------|---------|-----------|----------|---------|
| 0 | 12347.0    | 1       | 5         | 2540.29  | 0       |
| 1 | 12348.0    | 248     | 1         | 367.00   | 1       |
| 2 | 12350.0    | 309     | 1         | 334.40   | 1       |
| 3 | 12352.0    | 35      | 5         | 456.08   | 0       |
| 4 | 12355.0    | 213     | 1         | 459.40   | 1       |

Cluster Summary (Mean RFM values):

|   | Cluster | Recency    | Frequency | Monetary      |
|---|---------|------------|-----------|---------------|
| 0 | 0       | 43.869005  | 3.345539  | 1054.107942   |
| 1 | 1       | 256.334347 | 1.509625  | 439.610284    |
| 2 | 2       | 0.500000   | 29.000000 | 111762.715000 |
| 3 | 3       | 13.574468  | 27.787234 | 17761.548085  |

Customer Count in Each Cluster:

|   | Cluster | CustomerCount |
|---|---------|---------------|
| 0 | 0       | 1939          |
| 1 | 1       | 987           |
| 2 | 2       | 2             |
| 3 | 3       | 47            |

## 6. Marketing Recommendations:

To maximize revenue and improve customer retention, the business can develop targeted marketing strategies for each customer segment identified through RFM segmentation. Here are some actionable marketing recommendations based on typical characteristics of each segment:

### 1. High-Value & Engaged Customers (Cluster 1 - Example: Low Recency, High Frequency, High Monetary)

**Characteristics:** Although they may not have made a purchase recently, these customers are very engaged, spend a lot of money, and make frequent purchases.

**Marketing Plan:**

Personalized Offers: To promote recurring business and re-engagement, offer customized discounts or loyalty benefits.

Exclusive Access: To reward enduring loyalty, provide premium services, early access to sales, or exclusive merchandise.

Win-Back Initiatives: They are valuable clients even though they are not very recent. To entice them back, provide customized deals in targeted win-back emails.

### 2. Loyal but Inactive Customers (Cluster 2 - Example: High Recency, Moderate Frequency, Moderate Monetary)

**Characteristics:** Despite having recently made a purchase, these customers have only made relatively frequent and expensive purchases.

**Marketing Strategy:**

Engagement Campaigns: To keep them engaged, send them interesting information like newsletters, product recommendations, or a reminder about their most recent purchase.

Referral Programs: By providing discounts to both the referrer and the referee, you may increase the frequency of purchases and encourage them to recommend others.

Reinforce Value Proposition: Through tailored emails that highlight the advantages and reviews of your products, remind them of the value they receive from them.

### 3. At-Risk Customers (Cluster 3 - Example: High Recency, Low Frequency, Low Monetary)

**Characteristics:** consumers who have recently made purchases but have not spent a lot of money or made frequent purchases. These clients might be in danger of losing interest.

**Marketing Strategy:**

Targeted Discounts: To promote recurring business, provide time-sensitive discounts. Use free shipping or bundled products to make the offer more alluring.

Cross-selling and upselling: Provide upgrades or related products depending on their most recent purchase. Utilize recommendations based on data to raise the average order value.

Customer input: Get input from customers to find out why they don't spend much money, then utilize that information to provide tailored solutions (e.g., new product features or services).

#### 4. Low-Engaged Customers (Cluster 4 - Example: High Recency, Low Frequency, High Monetary)

**Characteristics:** Despite not making many purchases, these clients have spent a lot of money. Although their recent purchases show enthusiasm, they require encouragement to participate more.

##### Marketing Strategy:

Loyalty Programs: Put in place or advertise a program where consumers receive points for every purchase, which incentivizes them to make larger and more frequent purchases.

Email Marketing Campaigns for New Products: Inform them of any new features, services, or goods that could encourage them to make more regular purchases.

Exclusive Offers for Recurring Customers: Encourage recurring business by providing special offers or discounts to loyal customers.

##### Overall Suggestions for Every Segment:

Email Marketing Based on Segmentation: Create individualized product suggestions for regular customers and re-engagement campaigns for high-risk clients by customizing email messages according to the behavior of each category.

Automated Workflows: Create automated email campaigns for re-engaging dormant clients, post-purchase follow-ups, or abandoned carts.

Focus on Retention and Upselling: By sustaining engagement and loyalty through customized offerings, always strive to keep clients in high-value clusters. Concentrate on tactics that promote more frequent and expensive purchases from lower-frequency clients.

Track Cluster Shifts: Monitor the flow of clients between clusters over time. You can use bespoke campaigns to retarget high-value segments, for instance, if they stop being active.

## 7. Visualization:

### Heat Map:

In order to comprehend the links between these variables, this analysis uses a heatmap to display the correlation between Recency, Frequency, and Monetary (RFM) measurements. First, `rfm[['Recency', 'Frequency', 'Monetary']]` is used to retrieve the pertinent RFM metrics from the dataset. The Pearson correlation coefficients between the RFM variables are then computed using the `corr()` function, which also calculates the correlation matrix.

The correlation matrix is then plotted as a heatmap using the `seaborn.heatmap()` function. The `annot=True` option shows the correlation values within each cell, and the `cmap='coolwarm'` color palette is used to visually differentiate between positive and negative correlations. The color scale range is defined to depict correlation values between -1 (strong negative correlation) and 1 (strong positive correlation) via the `vmin=-1` and `vmax=1` parameters. For context, a plot title is added using the `plt.title()` function. Understanding consumer behavior for focused

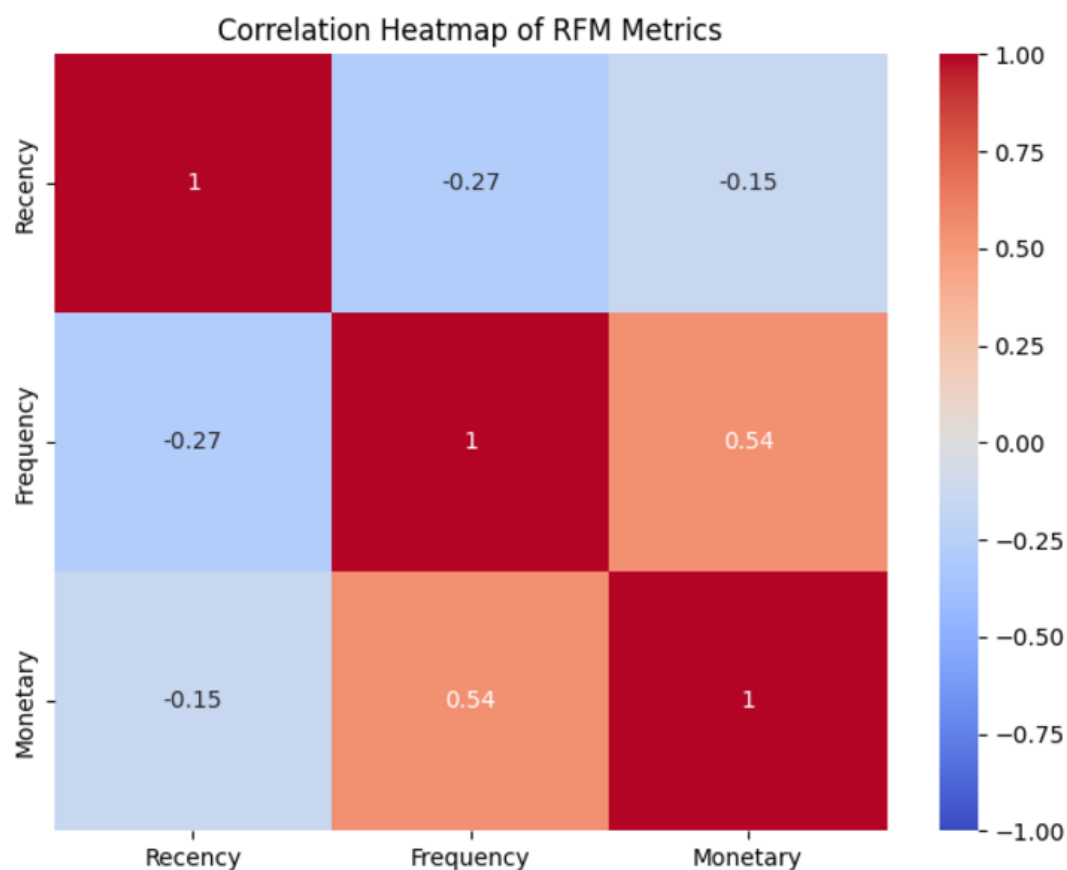
marketing tactics is made easier with the help of this visualization, which shows the relationships between each RFM measure.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate the correlation matrix for RFM metrics
rfm_metrics = rfm[['Recency', 'Frequency', 'Monetary']]
correlation_matrix = rfm_metrics.corr()

# Plot the heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Heatmap of RFM Metrics')
plt.show()
```

## Result:



## 3D Scatter plot:

This analysis visualizes customer segments in a 3D scatter plot to better understand the distribution of Recency, Frequency, and Monetary metrics for each segment. The visualization is created using matplotlib's 3D plotting capabilities.

The ideal number of clusters, as determined by optimal\_clusters, is used to segment the customer data. The scatter() function is used to plot each cluster as a scatter plot, with the X, Y, and Z axes representing the Recency, Frequency, and Monetary metrics, respectively. Each

cluster is given a suitable label with `label=f'Segment {cluster}'`, and the clusters are color-coded using the viridis palette and the `sns.color_palette()` method.

The `plt.title()` adds a title to the plot for clarity, while the `ax.set_xlabel()`, `ax.set_ylabel()`, and `ax.set_zlabel()` functions label the axes. Lastly, the cluster names are shown in the plot using `plt.legend()`, which makes it simple to discern between the various customer categories. With the help of this 3D scatter plot, which shows how different consumer segments differ in terms of their RFM, marketing and business strategies may be more precisely targeted.

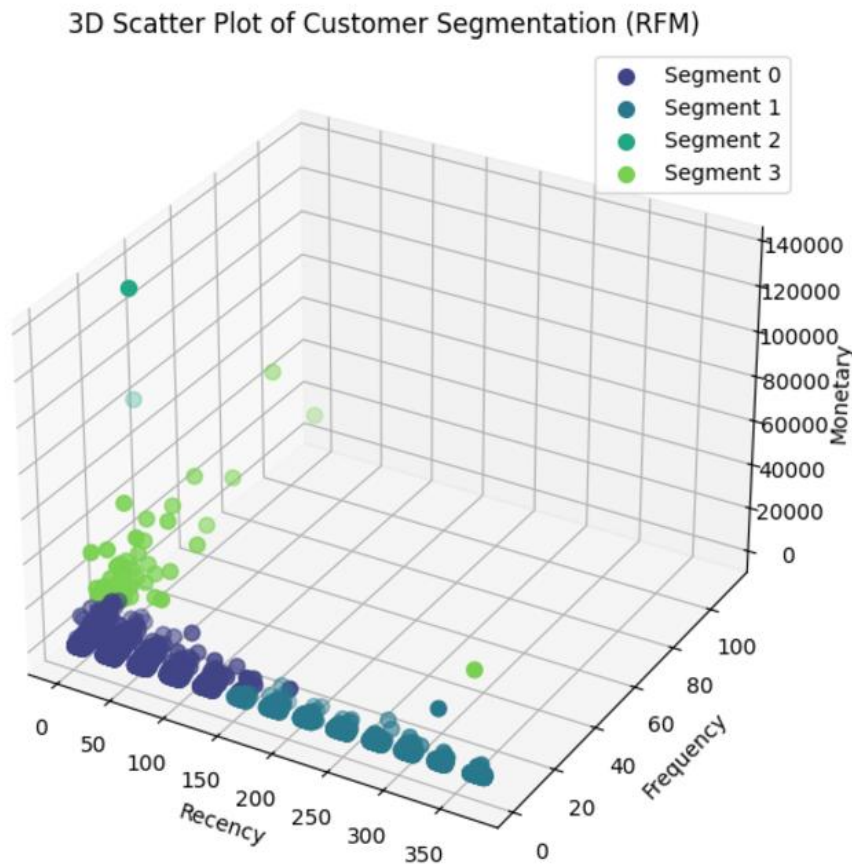
```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
colors = sns.color_palette('viridis', n_colors=optimal_clusters)

for cluster in range(optimal_clusters):
    cluster_data = rfm[rfm['Cluster'] == cluster]
    ax.scatter(cluster_data['Recency'], cluster_data['Frequency'], cluster_data['Monetary'],
               label=f'Segment {cluster}', color=colors[cluster], s=50)

ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
plt.title('3D Scatter Plot of Customer Segmentation (RFM)')
plt.legend()
plt.show()
```

## Result:



## Analysis:

### 1. Data Overview:

Print(f"Dataset Size: {ed.shape}") shows the number of rows and columns in the dataset in a readable format, which is obtained using the shape attribute. This stage enables the user to rapidly determine the dataset's size, which is crucial for figuring out how much data they will have to deal with. It can also assist in spotting possible problems with the data, including missing values or mismatched dimensions, before additional analysis is done.

```
import pandas as pd
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
print(f"Dataset Size: {ed.shape}")
```

## Result:

---

Dataset Size: (541909, 8)

## Reviewing Column Names and Descriptions:

"info" section is a valuable inclusion in a report because it provides a concise yet comprehensive summary of the dataset, offering insights into data types, missing values, memory usage, and data quality. It aids in efficient data exploration and supports informed decision-making during data analysis and preparation.

Column names are often self-explanatory, but when working with complex datasets, it's helpful to review any available descriptions or metadata associated with the columns. This information can provide context and help in understanding the meaning of each column.

```
print("\nColumn Descriptions:")
print(ed.info())

print("\nSummary Statistics for Numerical Columns:")
print(ed.describe())
```

## Result:

```
Column Descriptions:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   InvoiceNo       541909 non-null object  
 1   StockCode      541909 non-null object  
 2   Description    540455 non-null object  
 3   Quantity       541909 non-null int64  
 4   InvoiceDate    541909 non-null object  
 5   UnitPrice      541909 non-null float64  
 6   CustomerID     406829 non-null float64  
 7   Country        541909 non-null object  
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
None

Summary Statistics for Numerical Columns:

```

|       | Quantity      | UnitPrice     | CustomerID    |
|-------|---------------|---------------|---------------|
| count | 541909.000000 | 541909.000000 | 406829.000000 |
| mean  | 9.552250      | 4.611114      | 15287.690570  |
| std   | 218.081158    | 96.759853     | 1713.600303   |
| min   | -80995.000000 | -11062.060000 | 12346.000000  |
| 25%   | 1.000000      | 1.250000      | 13953.000000  |
| 50%   | 3.000000      | 2.080000      | 15152.000000  |
| 75%   | 10.000000     | 4.130000      | 16791.000000  |
| max   | 80995.000000  | 38970.000000  | 18287.000000  |

### **DataSet Time Period:**

In this step, we ensured that the InvoiceDate column was converted to datetime format to facilitate accurate date-based analysis. We then determined the start date and end date of the dataset by identifying the earliest and latest dates in the InvoiceDate column, respectively. By doing so, we were able to ascertain the period covered by the dataset, which helps in understanding the time span of the transaction data. The start and end dates provide valuable context when analyzing trends and patterns over time.

```
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
start_date = ed['InvoiceDate'].min()
end_date = ed['InvoiceDate'].max()

print("Period covered by the dataset:")
print("Start Date:", start_date)
print("End Date:", end_date)
```

### **Result:**

```
Period covered by the dataset:
Start Date: 2010-12-01 08:26:00
End Date: 2011-12-09 12:50:00
```

## **2. Customer Analysis:**

### **Unique Customers:**

In this part of the analysis, we determined the number of unique customers in the dataset by applying the `nunique()` function to the CustomerID column. This function identifies and counts distinct customer IDs, offering insight into how many individual customers have made purchases. This metric is crucial for assessing customer base size and can inform various business strategies, such as targeting customer retention or growth efforts. The total number of unique customers was then displayed to provide an understanding of the diversity of the customer base in the dataset.

```
unique_customers = ed['CustomerID'].nunique()
print("Number of unique customers:", unique_customers)
```

### **Result:**

```
Number of unique customers: 4372
```

## Distribution of Number of orders per Customer:

This section of the analysis examines the distribution of order frequency per customer in the e-commerce dataset, with a specific focus on identifying patterns in customer ordering behavior. The analysis follows these steps:

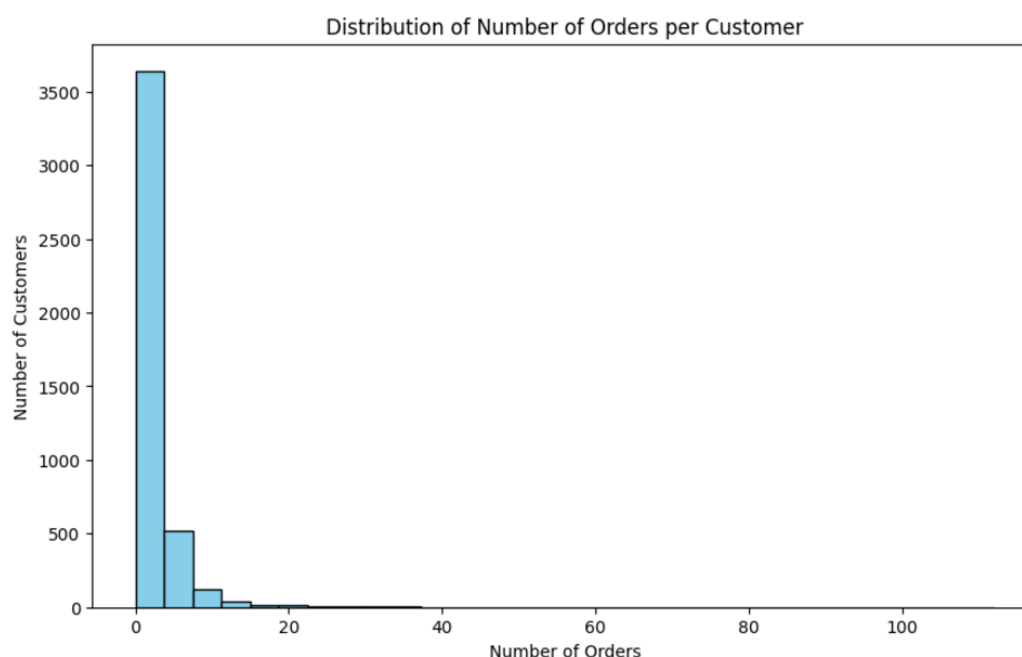
Grouping Data by Customer ID: We started by counting the number of distinct orders (invoices) for every client after classifying the data by CustomerID. This was accomplished by using the `nunique()` method to the InvoiceDate column, which provides the number of unique orders placed by each client, and the `groupby()` function to the CustomerID column.

Order Count Distribution: To see the distribution of order counts, we computed the number of distinct orders for each customer and then produced a histogram. By showing trends in the customer base's purchasing patterns, the histogram assists us in understanding the proportion of customers who fit into certain order frequency groups.

Histogram Plotting: The histogram, which displays the number of consumers that placed a specific quantity of orders, was produced with 30 bins using Matplotlib. In order to improve visual clarity, the plot was customized with black margins and sky-blue bars. To make the material easily comprehensible, the plot was labeled with the proper names and axes. This study offers insightful information about consumer behavior, assisting in determining if the majority of consumers make infrequent purchases or are regular buyers. These kinds of insights are essential for customizing marketing tactics for various clientele groups.

```
# Group by customer ID and count the number of unique orders per customer
order_counts = ed.groupby('CustomerID')['InvoiceDate'].nunique()
# Plot the distribution of order counts per customer
plt.figure(figsize=(10, 6))
order_counts.plot(kind='hist', bins=30, color='skyblue', edgecolor='black')
plt.title("Distribution of Number of Orders per Customer")
plt.xlabel("Number of Orders")
plt.ylabel("Number of Customers")
plt.show()
```

### Result:



### Top 5 Customers by Order Count:

In this analysis, we first grouped the data by customer ID using the `groupby()` function and counted the number of unique orders per customer by applying `nunique()` on the `InvoiceNo` column. This allowed us to understand how many distinct orders each customer placed. Then, we identified the top 5 customers who made the most purchases by applying the `nlargest(5)` function to the resulting order counts. This helped us highlight the highest-purchasing customers, which can be valuable for personalized marketing or loyalty programs. The top 5 customers were displayed based on their order frequency.

```
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
# Group by customer ID and count the number of unique orders per customer
order_counts = ed.groupby('CustomerID')['InvoiceNo'].nunique()

# Identify the top 5 customers with the most purchases
top_customers = order_counts.nlargest(5)
print("Top 5 customers with the most purchases:")
print(top_customers)
```

### Result:

```
Top 5 customers with the most purchases:
CustomerID
14911.0    248
12748.0    224
17841.0    169
14606.0    128
13089.0    118
Name: InvoiceNo, dtype: int64
```

## 3. Product Analysis:

Counting Product Frequency: The goal of this research is to determine which items in the e-commerce dataset are most commonly bought. The `StockCode` column, which gives the number of times each product was bought, is subjected to the `value_counts()` function. This enables us to ascertain how frequently each product is purchased, providing us with information about consumer trends and preferences. This phase produces a sequence of values that show how many times each product was purchased, with the index representing product codes.

Top 10 Most Frequently Purchased Products: Finding the top ten most often bought products comes next, after the frequency of each product has been determined. The `head(10)` function on the `product_counts` series, which yields the first 10 entries, is used to do this. After that, these top products are printed for convenient access. Businesses can gain a better understanding of client demand and make well-informed decisions regarding inventory, marketing, and sales strategies by concentrating on the most often purchased items.

```
# Count the frequency of each product being purchased
product_counts = ed['StockCode'].value_counts()
# Get the top 10 most frequently purchased products
top_products = product_counts.head(10)
print("Top 10 most frequently purchased products:")
print(top_products)
```

## Result:

```
Top 10 most frequently purchased products:
StockCode
85123A    2313
22423     2203
85099B    2159
47566     1727
20725     1639
84879     1502
22720     1477
22197     1476
21212     1385
20727     1350
Name: count, dtype: int64
```

## Average Product Price in the Dataset:

In this analysis, we calculated the average price of products by applying the `mean()` function to the `UnitPrice` column of the dataset. This gives us the mean price of all products sold in the dataset, providing insights into the general pricing strategy or customer purchasing behavior. The result was then displayed, allowing us to understand the typical price point of products offered in the store. This information can be useful for inventory pricing strategies or sales trend analysis.

```
# Calculate the average price of products
average_price = ed['UnitPrice'].mean()
print("Average price of products:", average_price)
```

## Result:

```
Average price of products: 4.611113626088513
```

## Product Category with highest Revenue:

Calculating Revenue: This stage involves figuring out how much money was made from each transaction in the dataset. Each row's revenue is calculated by multiplying the product's unit price by the quantity sold. The formula `ed['revenue'] = ed['UnitPrice'] * ed['Quantity']` is used to make this computation, and the revenue that results for each product transaction is saved in a new column named 'revenue'. This enables us to later add together all of the money made from each product or category.

Grouping by Product Category: The data is then grouped by product category, and the total revenue for each category is determined once the revenue for each transaction has been determined. The `groupby('Description')` function is used to do this, grouping the data according to the Description column, which represents product categories. The total income produced by each category is then obtained by using the `sum()` method to the 'revenue' column. This gives organizations a consolidated picture of revenue by product category, enabling them to determine which categories account for the largest portion of total revenue.

Identifying the Top Category: Finding the product category that brought in the most money is the last stage. The `max()` function returns the entire revenue for the category that has the highest revenue, while the `idxmax()` function determines which category has the highest revenue. The `print()` method is used to print the results, showing the category along with the income associated with it. The product category with the highest revenue performance is highlighted in this research, offering insightful information for strategic choices about resource allocation, inventories, and promotions.

```
# Calculate revenue for each row
ed['revenue'] = ed['UnitPrice'] * ed['Quantity']
# Group by product category and sum up the revenue for each category
category_revenue = ed.groupby('Description')['revenue'].sum()
# Identify the category with the highest revenue
top_category = category_revenue.idxmax()
highest_revenue = category_revenue.max()

print("Product category with the highest revenue:")
print("Category:", top_category)
print("Revenue:", highest_revenue)
```

## Result:

```
Product category with the highest revenue:
Category: DOTCOM POSTAGE
Revenue: 206245.48
```

## 4. Time Analysis:

### Orders by Day of the Week:

We looked at the distribution of orders on various days of the week in our investigation. We used the `groupby()` and `size()` methods to count the number of orders for each day after extracting the day of the week from the InvoiceDate column and grouping the data by DayOfWeek. The amount of orders each day was then graphically displayed by plotting the findings using a bar plot. By identifying the days with the highest order volume, this study gives firms important information for maximizing staff scheduling, promotions, and focused marketing campaigns.

### Orders by Hour of the Day:

We then looked at how orders were distributed throughout the day. The quantity of orders at each time was determined by grouping the data by hour after extracting the hour of the day from the InvoiceDate column. To see how orders change during the day, a line plot was made. Businesses may determine when consumers are most engaged and when to devote resources efficiently by using this information, which shows peak order hours. Businesses may optimize their operational strategies, such as modifying shipment timetables or introducing time-specific promotions, by comprehending order patterns by time.

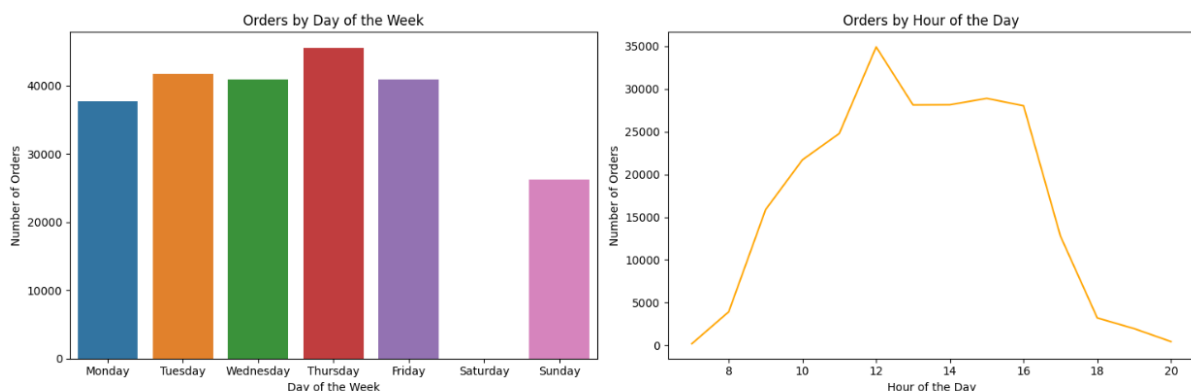
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
ed = pd.read_csv(r"C:\project2\IE6400\ecommercedata.csv")
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'], errors='coerce')
# Assume `ed` is your DataFrame and has a column 'InvoiceDate' with datetime information
# Extract day of the week and hour from 'InvoiceDate'
ed['DayOfWeek'] = ed['InvoiceDate'].dt.day_name()
ed['Hour'] = ed['InvoiceDate'].dt.hour
# Group by day of the week to see order patterns by day
day_counts = ed.groupby('DayOfWeek').size().reindex(
    ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
)
# Group by hour to see order patterns by time of day
hour_counts = ed.groupby('Hour').size()

# Plotting the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
# Plot day of week without the deprecated palette usage
sns.barplot(x=day_counts.index, y=day_counts.values, ax=axes[0], hue=day_counts.index, dodge=False, legend=False)
axes[0].set_title('Orders by Day of the Week')
axes[0].set_xlabel('Day of the Week')
axes[0].set_ylabel('Number of Orders')

# Plot hour of day
sns.lineplot(x=hour_counts.index, y=hour_counts.values, ax=axes[1], color='orange')
axes[1].set_title('Orders by Hour of the Day')
axes[1].set_xlabel('Hour of the Day')
axes[1].set_ylabel('Number of Orders')

plt.tight_layout()
plt.show()
```

### Result:



## Average Order processing Time:

Calculating Time Between Orders: To preserve chronological order, we first make sure the InvoiceDate column is in datetime type before sorting the dataset by CustomerID and InvoiceDate. We determine the time difference between each customer's successive orders using groupby() and diff(). The result is saved in the TimeDiff column as the number of days between orders.

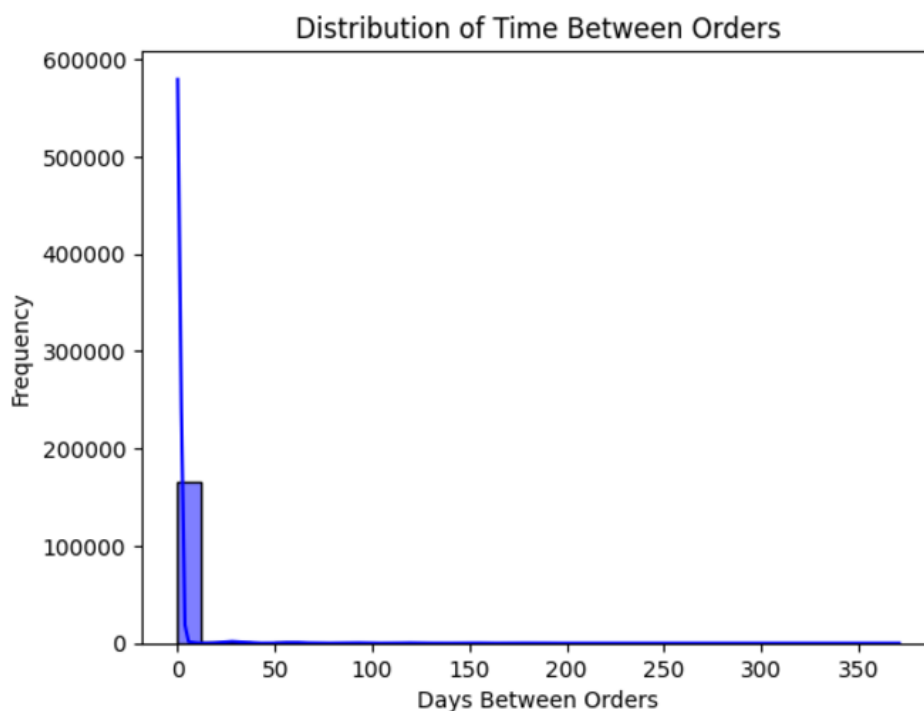
Calculating Average Time Between Orders: The TimeDiff column's mean is used to calculate the average time between orders. The distribution of these time differences is displayed using a histogram with Kernel Density Estimation (KDE), which enables us to examine the frequency of different order intervals. This measure gives information about the average interval between client transactions.

```
# Ensure 'InvoiceDate' is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Sort data by CustomerID and InvoiceDate
ed = ed.sort_values(by=['CustomerID', 'InvoiceDate'])
# Calculate time difference between consecutive orders for each customer
ed['TimeDiff'] = ed.groupby('CustomerID')['InvoiceDate'].diff().dt.days
# Calculate the average processing time (ignoring NaN for the first order of each customer)
avg_processing_time = ed['TimeDiff'].mean()

# Display the result
print(f"Average Time Between Orders: {avg_processing_time:.2f} days")
sns.histplot(ed['TimeDiff'].dropna(), bins=30, kde=True, color='blue')
plt.title('Distribution of Time Between Orders')
plt.xlabel('Days Between Orders')
plt.ylabel('Frequency')
plt.show()
```

## Result:

Average Time Between Orders: 1.83 days



## Seasonal Trends:

Patterns or variations that recur over predetermined time periods, such months, quarters, or seasons (spring, summer, fall, and winter), are referred to as seasonal trends. These patterns are crucial for comprehending consumer behavior, sales cycles, or demand fluctuations as they are frequently impacted by outside variables such as holidays, weather, or cultural events. Businesses may predict peak times and modify their marketing, inventory, and personnel plans by analyzing seasonal trends. These trends may be successfully highlighted by visualizations such as line charts or bar plots organized by season, which improve long-term planning and provide insights for decision-making.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Ensure InvoiceDate is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])

# Extract temporal features
ed['Year'] = ed['InvoiceDate'].dt.year
ed['Month'] = ed['InvoiceDate'].dt.month
ed['DayOfWeek'] = ed['InvoiceDate'].dt.dayofweek
ed['MonthName'] = ed['InvoiceDate'].dt.month_name()
ed['DayName'] = ed['InvoiceDate'].dt.day_name()

# Define seasons based on months
def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'

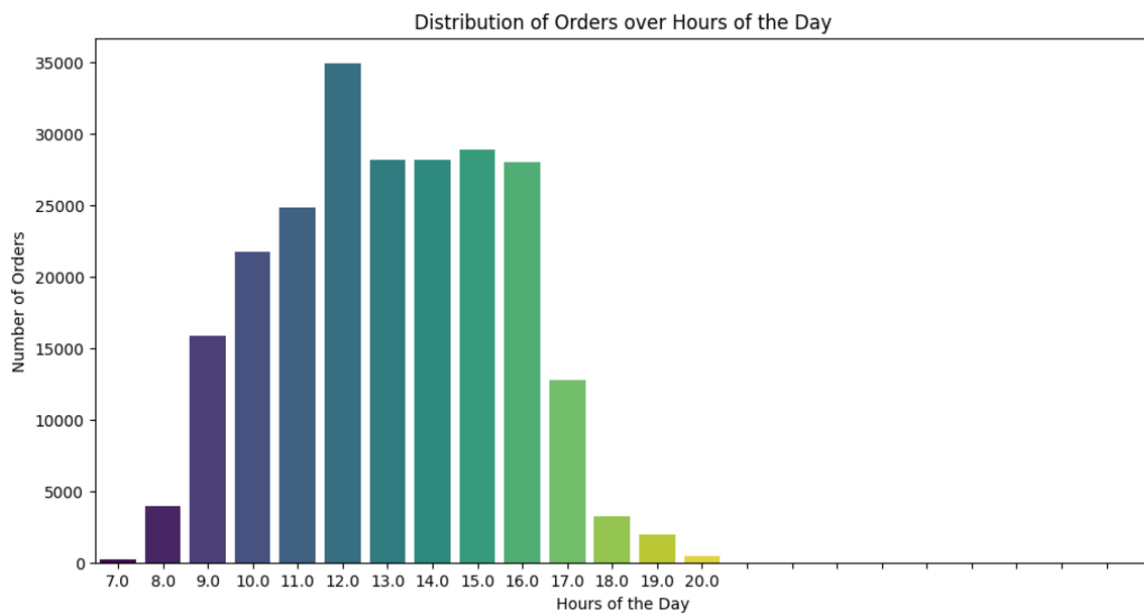
ed['Season'] = ed['Month'].apply(get_season)
```

### Distribution of Orders over hours of the day:

The first step in this analysis is to create a new HoursOfDay column by converting the InvoiceDate column to datetime format and using the .dt.hour function to extract the hour of the day. The hour at which each order was placed is shown in this column. We utilize Seaborn's countplot() method, which shows the amount of orders for each hour, to see the distribution of orders throughout the day. The x-axis is resized to display all 24 hours, and the viridis color scheme is used. This graphic provides insightful information about consumer ordering trends and highlights periods of high order volume.

```
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Create 'HoursOfDay' column
ed['HoursOfDay'] = ed['InvoiceDate'].dt.hour
# Plot the distribution of orders over hours of the day
plt.figure(figsize=(12, 6))
sns.countplot(x='HoursOfDay', data=ed, hue='HoursOfDay', legend=False, palette='viridis')
plt.title('Distribution of Orders over Hours of the Day')
plt.xlabel('Hours of the Day')
plt.ylabel('Number of Orders')
plt.xticks(range(24)) # Ensure all hours are shown on the x-axis
plt.show()
```

## Result:

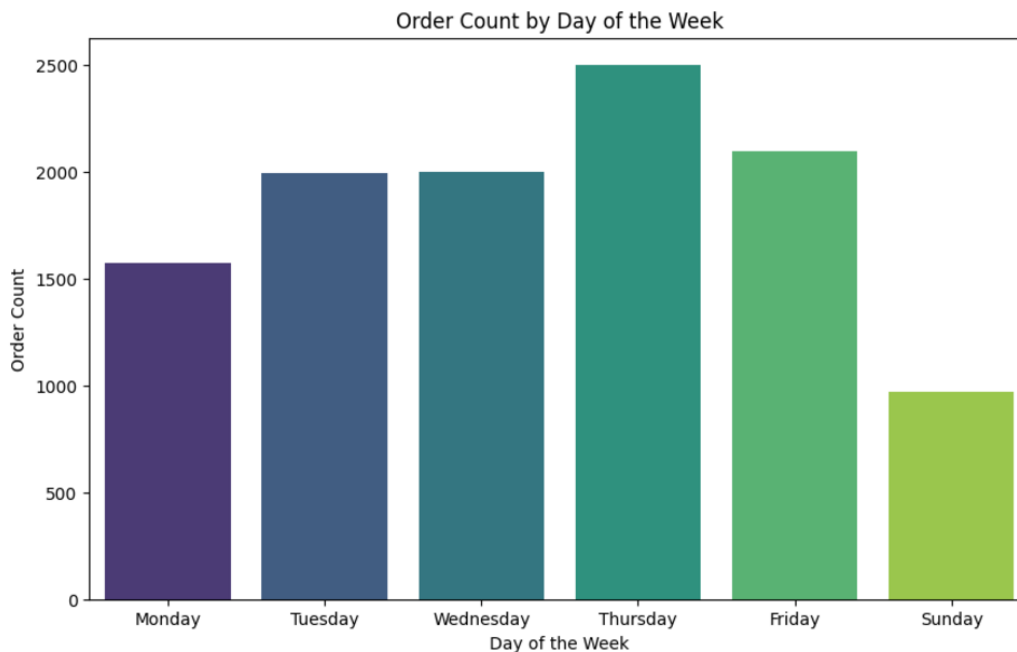


## Order count by day of the week:

To look at order patterns, we aggregate the data by day of the week in this study. Groupby() is used to group the data, and the agg() method is used to get the total number of goods (Quantity) and the number of unique orders (InvoiceNo). For clarity, the columns are then renamed OrderCount and TotalQuantity. To translate numerical day indices into real weekday names, a day mapping is used. The order count per day is then shown using a bar plot created with Seaborn's barplot(), which offers insights into weekly order trends and consumer behavior.

```
# Aggregate data by day of the week
weekly_data = ed.groupby('DayOfWeek').agg({'InvoiceNo': 'nunique', 'Quantity': 'sum'}).reset_index()
weekly_data = weekly_data.rename(columns={'InvoiceNo': 'OrderCount', 'Quantity': 'TotalQuantity'})
# Map day indices to names
day_mapping = {0: 'Monday', 1: 'Tuesday', 2: 'Wednesday', 3: 'Thursday', 4: 'Friday', 5: 'Saturday', 6: 'Sunday'}
weekly_data['DayName'] = weekly_data['DayOfWeek'].map(day_mapping)
# Plot weekly trends
plt.figure(figsize=(10, 6))
sns.barplot(data=weekly_data, x='DayName', y='OrderCount', hue='DayName', palette='viridis')
plt.title('Order Count by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Order Count')
plt.show()
```

## Result:

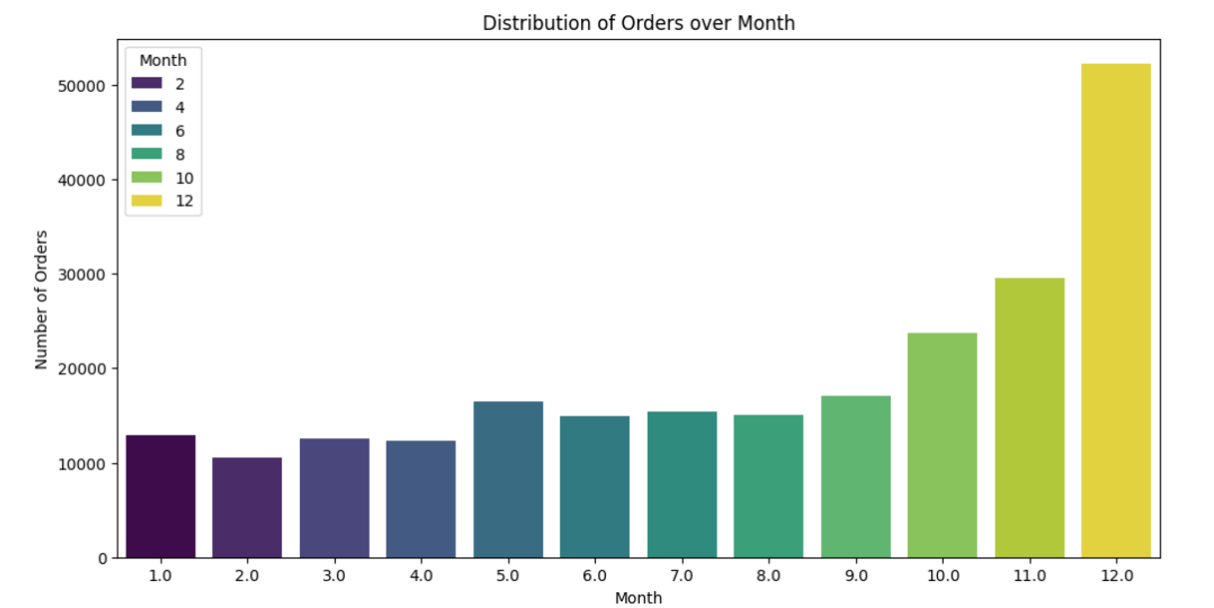


### Distribution of order per month:

We start our analysis by using `pd.to_datetime()` to make sure the `InvoiceDate` field is in datetime format. Next, we add a new `Month` column and use `dt.month` to extract the month from the `InvoiceDate` column. This enables us to see how orders are distributed throughout several months. The amount of orders for each month is plotted using Seaborn's `countplot()`, with the `viridis` color scheme used for clarity. The resultant bar chart provides important insights into how orders vary throughout the year by highlighting seasonal changes in order volume.

```
# Convert 'InvoiceDate' to datetime if it's not already
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Extract the 'Month' column (optional if already exists)
ed['Month'] = ed['InvoiceDate'].dt.month
# Plot the distribution of orders over months
plt.figure(figsize=(12, 6))
sns.countplot(x='Month', hue='Month', data=ed, palette='viridis')
plt.title('Distribution of Orders over Month')
plt.xlabel('Month')
plt.ylabel('Number of Orders')
plt.show()
```

## Result:

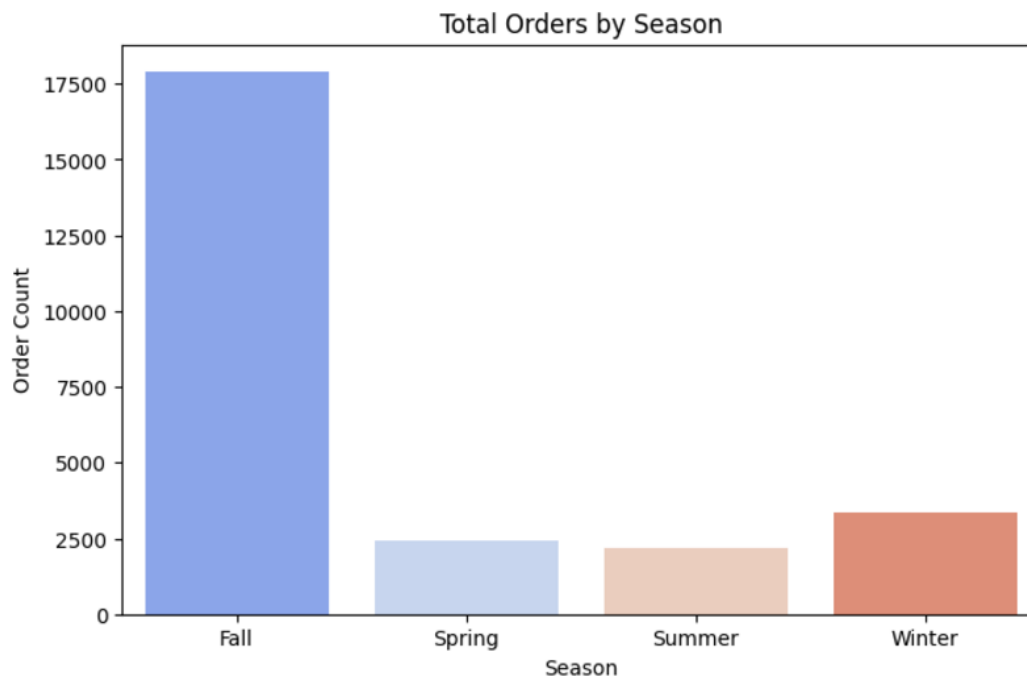


### Total order by season:

To find seasonal trends in client orders, we aggregate the data by season in this study. The number of distinct orders (InvoiceNo) and the total number of products (Quantity) are determined using the `agg()` function after the data is categorized using `groupby()` by the Season column. For clarity, the columns are now called OrderCount and TotalQuantity. After that, a bar plot with the coolwarm color scheme is made using Seaborn's `barplot()` to show the total number of orders for each season. This highlights seasonal variations in order volume and helps identify trends in consumer behavior.

```
# Aggregate data by season
seasonal_data = ed.groupby('Season').agg({'InvoiceNo': 'nunique', 'Quantity': 'sum'}).reset_index()
seasonal_data = seasonal_data.rename(columns={'InvoiceNo': 'OrderCount', 'Quantity': 'TotalQuantity'})
# Plot seasonal trends
plt.figure(figsize=(8, 5))
sns.barplot(data=seasonal_data, x='Season', y='OrderCount', hue='Season', palette='coolwarm')
plt.title('Total Orders by Season')
plt.xlabel('Season')
plt.ylabel('Order Count')
plt.show()
```

**Result:**



## 5. Geographical Analysis:

**Top countries with highest number of orders:**

Using the `groupby()` method on the `nation` column, we first count the number of unique orders per nation. Next, we apply `nunique()` to the `InvoiceNo` to obtain the number of distinct orders per country. For clarity, the resultant column is now called `OrderCount`. The nations with the highest order volumes are then determined by sorting the data in descending order. We choose the top 5 nations with the most orders by using `head(5)`. These nations are shown in the final result, which offers information on the areas with the highest levels of consumer activity.

```
# Count the number of unique orders per country
country_order_count = ed.groupby('Country')['InvoiceNo'].nunique().reset_index()
# Rename columns for clarity
country_order_count = country_order_count.rename(columns={'InvoiceNo': 'OrderCount'})
# Sort the countries by the number of orders in descending order
top_5_countries = country_order_count.sort_values(by='OrderCount', ascending=False).head(5)
# Reset the index for clean output
top_5_countries = top_5_countries.reset_index(drop=True)

# Display the result
print("Top 5 Countries with the Highest Number of Orders:")
print(top_5_countries.to_string(index=False))
```

## Result:

Top 5 Countries with the Highest Number of Orders:

| Country        | OrderCount |
|----------------|------------|
| United Kingdom | 23494      |
| Germany        | 603        |
| France         | 461        |
| EIRE           | 360        |
| Belgium        | 119        |

---

## Correlation Analysis:

### Average Order Value (AOV) by Country Analysis:

In this section, we focused on analyzing the average order value (AOV) across different countries. The primary goal was to examine how the average value of orders varies by country, providing insights into customer purchasing behavior. The following steps outline the methodology of this analysis:

Data Preparation: Making sure the InvoiceDate field was correctly transformed into a datetime format was the first step. Accurate time-based aggregations were made possible by this. The total value of each transaction was then determined by multiplying the Quantity and UnitPrice for each order to create a new column called TotalPrice.

Data Aggregation: To determine the overall cost and the quantity of distinct invoices for every nation, we combined the data by country using the groupby() method. Next, we divided the total price by the number of distinct invoices to get the AOV. For ease of presentation, the data was sorted by AOV to emphasize nations with higher order values.

Visualization and Interpretation: The print() method was used to present the findings, displaying the AOV for each nation. Using sns.barplot(), a bar plot was made to graphically depict the data, with the viridis color scheme used for clarity. Based on purchasing power, this graphic shows the top 10 nations with the greatest average order value (AOV), offering insightful information for focused marketing campaigns and company plans.

```

#Ensure proper data types
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Calculate Total Price for each order
ed['TotalPrice'] = ed['Quantity'] * ed['UnitPrice']
# Aggregate data: Calculate average order value (AOV) per country
country_aov = ed.groupby('Country').agg({'TotalPrice': 'sum', 'InvoiceNo': 'nunique'}).reset_index()
country_aov['AOV'] = country_aov['TotalPrice'] / country_aov['InvoiceNo']
# Sort by AOV for better visualization
country_aov = country_aov.sort_values(by='AOV', ascending=False)

# Display results
print("Average Order Value (AOV) by Country:")
print(country_aov[['Country', 'AOV']])

# Bar plot for AOV by country
plt.figure(figsize=(12, 8))
sns.barplot(data=country_aov.head(10), x='AOV', y='Country', hue='AOV', palette='viridis')
plt.title('Top 10 Countries by Average Order Value (AOV)')
plt.xlabel('Average Order Value (AOV)')
plt.ylabel('Country')
plt.show()

```

---

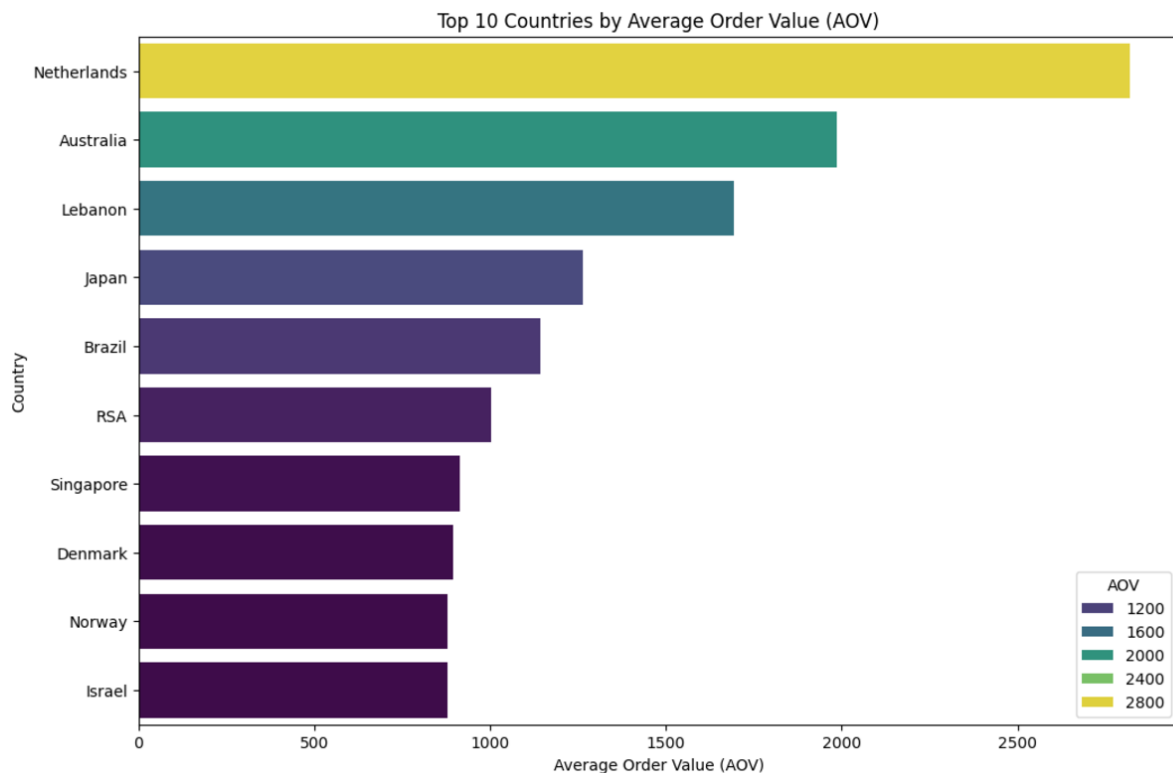
## Result:

```

Average Order Value (AOV) by Country:
   Country  AOV
24  Netherlands  2818.431089
0   Australia   1986.627101
21  Lebanon     1693.880000
20  Japan       1262.165000
4   Brazil      1143.600000
28  RSA         1002.310000
30  Singapore   912.039000
9   Denmark     893.720952
25  Norway      879.086500
18  Israel      878.646667
32  Sweden      795.563261
15  Greece      785.086667
33  Switzerland 761.964189
10  EIRE        731.324500
16  Hong Kong   674.469333
7   Cyprus      647.314500
35  United Arab Emirates 634.093333
17  Iceland     615.714286
5   Canada      611.063333
6   Channel Islands 608.675455
1   Austria     534.437895
31  Spain       521.662667
12  Finland     465.140417
13  France      428.208026
22  Lithuania   415.265000
27  Portugal    413.620000
14  Germany     367.658723
37  Unspecified 365.368462
36  United Kingdom 348.506272
3   Belgium     343.789580
19  Italy       307.100182
26  Poland      300.547500
11  European Community 258.350000
23  Malta       250.547000
34  USA         247.274286
8   Czech Republic 141.544000
2   Bahrain     137.100000
29  Saudi Arabia 65.585000

```

---



## 6. Customer Behavior:

### Customer Activity Duration Analysis:

In this section, we focused on analyzing the activity duration of customers, which is the time between their first and last purchase. The goal was to understand how long customers remain engaged with the business, providing insights into customer loyalty and retention. Here's the *breakdown of the analysis*:

Data Preparation: To enable accurate date-based computations, the first step was to make sure the InvoiceDate column was transformed into a datetime format using `pd.to_datetime()`. Additionally, as missing CustomerID values would not yield useful information about customer behavior, we eliminated any rows that had them.

Activity Calculation: The FirstPurchase and LastPurchase dates were then determined using the `min()` and `max()` functions, respectively, after the data had been grouped by CustomerID using `groupby()`. Each customer's ActiveDuration was determined by deducting the FirstPurchase from the LastPurchase, which produced the duration in days. We then calculated the average active duration across all customers to understand the typical length of customer engagement.

Visualization and Interpretation: The average active duration in days was shown using the `print()` function. Using `sns.histplot()`, a histogram was produced to show the distribution of active durations. A Kernel Density Estimate (KDE) was also included to help illustrate the

frequency and dispersion of active durations. Plotting the length of time clients usually remain active can help guide customer retention tactics.

```
# Ensure InvoiceDate is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Remove rows with null CustomerID
ed = ed.dropna(subset=['CustomerID'])
# Group by CustomerID to find first and last purchase dates
customer_activity = ed.groupby('CustomerID').agg(
    FirstPurchase=('InvoiceDate', 'min'),
    LastPurchase=('InvoiceDate', 'max')
).reset_index()

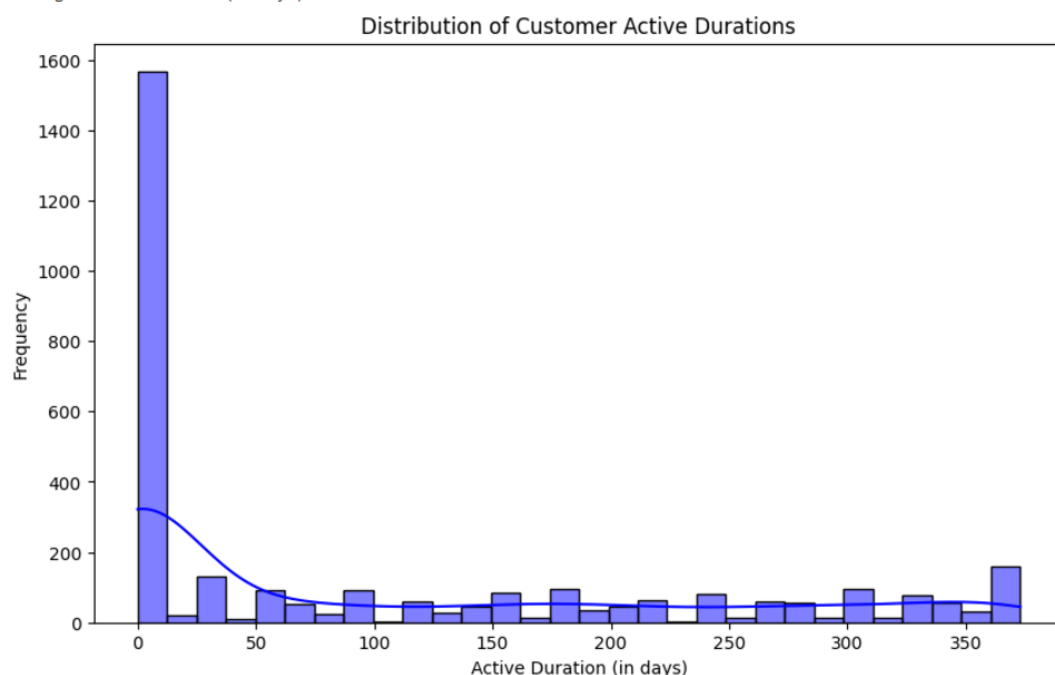
# Calculate active duration in days
customer_activity['ActiveDuration'] = (customer_activity['LastPurchase'] - customer_activity['FirstPurchase']).dt.days

# Calculate the average active duration
average_active_duration = customer_activity['ActiveDuration'].mean()

# Display the results
print("Average Active Duration (in days): {:.2f}".format(average_active_duration))
# Plot the distribution of active durations
plt.figure(figsize=(10, 6))
sns.histplot(customer_activity['ActiveDuration'], bins=30, kde=True, color='blue')
plt.title('Distribution of Customer Active Durations')
plt.xlabel('Active Duration (in days)')
plt.ylabel('Frequency')
plt.show()
```

## Result:

Average Active Duration (in days): 99.91



## Customer Segmentation Using RFM Analysis:

In this section, we applied RFM (Recency, Frequency, Monetary) analysis to segment customers into distinct groups based on their purchasing behavior. The goal was to identify

meaningful patterns in customer activity to enhance marketing strategies and improve customer retention. Below is a breakdown of the approach:

**Data Preparation:** Using `pd.to_datetime()`, we first made sure the `InvoiceDate` was in the proper datetime format. Next, we determined each customer's RFM metrics. The number of days since the last purchase was used to calculate recency. Each customer's unique invoice count was known as frequency, and each customer's total expenditure (product quantity multiplied by unit price) was known as monetary.

**Data Cleaning & Normalization:** Using `dropna()` and conditional filtering, we eliminated rows with incorrect or zero monetary values and made sure the RFM dataset had no missing or negative values. Prior to performing clustering, the data was scaled using `StandardScaler` to standardize the RFM metrics.

**Profiling and Clustering:** Using the normalized RFM measurements, we used K-Means clustering with four groupings. Each consumer was assigned to a cluster using the `fit_predict()` algorithm. In order to profile the customer groupings, we next compiled the attributes of each cluster, including customer numbers and mean values for Recency, Frequency, and Monetary.

**Visualization:** Lastly, we used a scatter plot with monetary on the y-axis and recency on the x-axis to show the clusters. By using `sns.scatterplot()` to give each cluster a unique color, we were able to visually examine the variations among the customer segments. This research offers a platform for adapting marketing activities to distinct target segments.

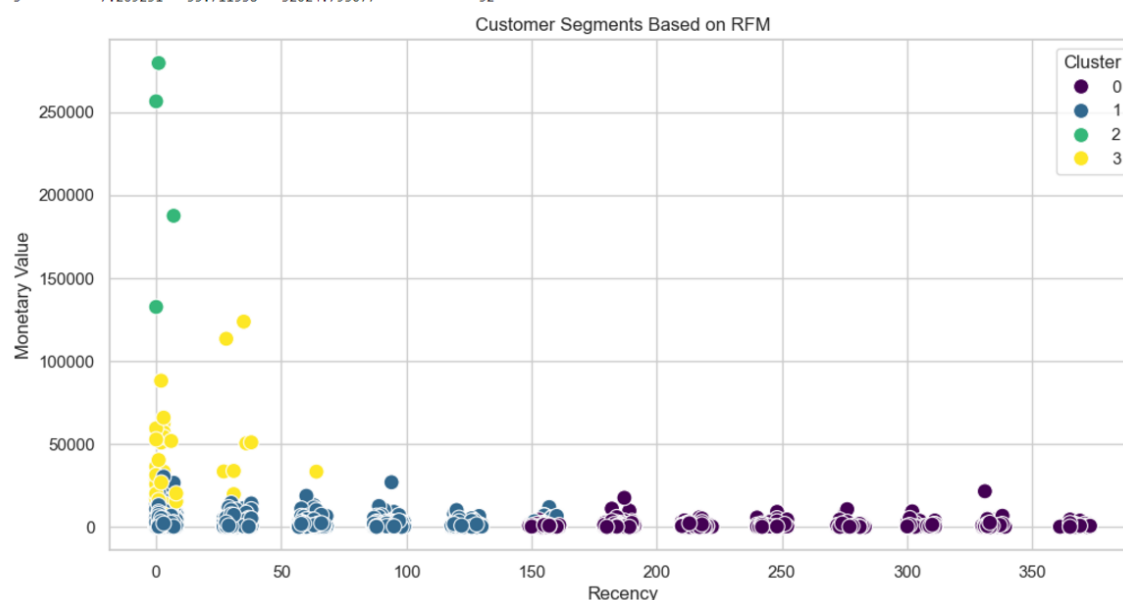
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Ensure InvoiceDate is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Calculate RFM metrics
reference_date = ed['InvoiceDate'].max()
rfm = ed.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda x: (reference_date - x.max()).days),
    Frequency=('InvoiceNo', 'nunique'),
    Monetary=('CustomerID', lambda x: (ed.loc[x.index, 'Quantity'] * ed.loc[x.index, 'UnitPrice']).sum())
).reset_index()
# Remove rows with invalid or zero monetary values
rfm = rfm[rfm['Monetary'] > 0]
# Check and handle missing or invalid values
rfm = rfm.dropna() # Drop rows with NaN values
rfm = rfm[(rfm[['Recency', 'Frequency', 'Monetary']] >= 0).all(axis=1)] # Keep non-negative values
# Normalize RFM metrics
scaler = StandardScaler()
rfm_normalized = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
# Apply K-Means clustering
kmeans = KMeans(n_clusters=4, random_state=42)
rfm['Cluster'] = kmeans.fit_predict(rfm_normalized)
# Profile clusters
cluster_summary = rfm.groupby('Cluster').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'CustomerID': 'count'
}).rename(columns={'CustomerID': 'CustomerCount'})
```

```
# Display results
print("Cluster Summary:")
print(cluster_summary)
# Visualize clusters
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="whitegrid")
plt.figure(figsize=(12, 6))
sns.scatterplot(data=rfm, x='Recency', y='Monetary', hue='Cluster', palette='viridis', s=100)
plt.title('Customer Segments Based on RFM')
plt.xlabel('Recency')
plt.ylabel('Monetary Value')
plt.show()
```

## Result:

| Cluster Summary: |            |            |               |               |
|------------------|------------|------------|---------------|---------------|
| Cluster          | Recency    | Frequency  | Monetary      | CustomerCount |
| 0                | 255.657469 | 3.286394   | 923.753788    | 1051          |
| 1                | 43.908954  | 6.660966   | 2082.181469   | 1988          |
| 2                | 2.000000   | 110.500000 | 213995.575000 | 4             |
| 3                | 7.269231   | 55.711538  | 32024.793077  | 52            |



## 7. Profitability Analysis:

### Profit generated by company:

Using `pd.to_datetime()`, we first made sure the `InvoiceDate` was in the proper datetime format before calculating the overall profit made by the business. Next, we multiplied the quantity by the unit price to determine the revenue for each order. All records' Revenue columns were added up to determine the overall profit, which was then shown. We then used `dt.to_period('M')` to group the data by YearMonth, which enabled us to add up the revenue for each month and get the monthly profit. Insights into the company's revenue trends were provided by the final output, which contained the total profit and monthly profit numbers.

```

# Ensure InvoiceDate is in datetime format
ed['InvoiceDate'] = pd.to_datetime(ed['InvoiceDate'])
# Calculate the total revenue (Quantity * UnitPrice)
ed['Revenue'] = ed['Quantity'] * ed['UnitPrice']
# Calculate the total profit
# Assuming gross revenue as profit
total_profit = ed['Revenue'].sum()

# Display the result
print("Total Profit Generated by the Company: {:.2f}".format(total_profit))
# Group by year-month
ed['YearMonth'] = ed['InvoiceDate'].dt.to_period('M')
# Calculate monthly profit
monthly_profit = ed.groupby('YearMonth')['Revenue'].sum().reset_index()
# Display the monthly profit
print("Monthly Profit:")
print(monthly_profit)

```

## Result:

Total Profit Generated by the Company: 8300065.81

Monthly Profit:

|    | YearMonth | Revenue   |
|----|-----------|-----------|
| 0  | 2010-12   | 353846.32 |
| 1  | 2011-01   | 199815.27 |
| 2  | 2011-02   | 163960.71 |
| 3  | 2011-03   | 203471.87 |
| 4  | 2011-04   | 180762.84 |
| 5  | 2011-05   | 260273.17 |
| 6  | 2011-06   | 234951.42 |
| 7  | 2011-07   | 183867.84 |
| 8  | 2011-08   | 281783.39 |
| 9  | 2011-09   | 276141.56 |
| 10 | 2011-10   | 395042.51 |
| 11 | 2011-11   | 450277.23 |
| 12 | 2011-12   | 342506.38 |

## Monthly Profit Trend Analysis:

In this section, we analyzed the monthly profit trend by visualizing the revenue over time. The goal was to understand how the revenue has changed month by month, which can provide valuable insights into seasonal fluctuations and overall business performance.

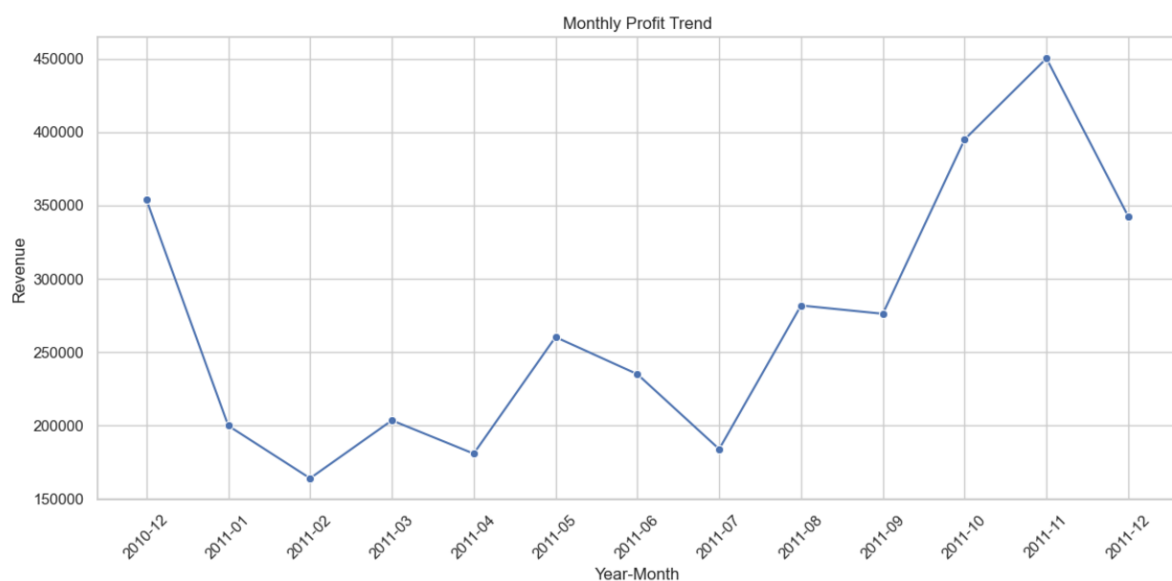
Data Preparation: Using `pd.to_numeric()`, we first made sure the Revenue column was numeric and that any non-convertible values were forced to NaN. Next, we used `dropna()` to eliminate rows in the Revenue column that had missing values. In order to properly depict the YearMonth column on the x-axis, we also used `astype(str)` to convert it to a string format.

Plotting: Using Seaborn's lineplot, we created a line chart that shows the revenue trend over time. The Year-Month values are represented on the x-axis, and the Revenue is represented on the y-axis. To make individual data points easier to see, we added markers to the line, and we rotated the x-axis labels for easier readability, particularly when working with month-year

combinations. The final plot gives a clear visual representation of how the revenue has changed over the months, making it easier to spot trends and patterns.

```
monthly_profit['Revenue'] = pd.to_numeric(monthly_profit['Revenue'], errors='coerce')
monthly_profit = monthly_profit.dropna(subset=['Revenue'])
monthly_profit['YearMonth'] = monthly_profit['YearMonth'].astype(str)
plt.figure(figsize=(12, 6))
sns.lineplot(data=monthly_profit, x='YearMonth', y='Revenue', marker='o')
plt.title('Monthly Profit Trend')
plt.xlabel('Year-Month')
plt.ylabel('Revenue')
plt.xticks(rotation=45) # Rotate x-axis labels for better understanding
plt.tight_layout()
plt.show()
```

## Result:



## Top products with high profits:

We initially multiplied the quantity and unit price for each order in our study to get the revenue for each transaction. After that, we used the Description column to categorize the data by product description. Then, we added up the revenue from all associated transactions to get the total revenue for each product. We arranged the total revenue data in descending order and chose the top five items to determine which ones had the most revenue. The end result showed the company's top five items, giving insight into which ones brought in the most money.

```

# Calculate the revenue for each transaction
ed['Revenue'] = ed['Quantity'] * ed['UnitPrice']
# Group by product description and calculate total revenue per product
product_revenue = ed.groupby('Description')['Revenue'].sum()
# Get the top 5 products with the highest revenue
top_5_products = product_revenue.sort_values(ascending=False).head(5)
# Display the results
top_5_products

```

## Result:

```

Description
REGENCY CAKESTAND 3 TIER          132870.40
WHITE HANGING HEART T-LIGHT HOLDER  93823.85
JUMBO BAG RED RETROSPOT           83236.76
PARTY BUNTING                    67687.53
POSTAGE                           66710.24
Name: Revenue, dtype: float64

```

## Conclusion

This report highlights the significance of using RFM analysis and clustering techniques to understand customer behavior in e-commerce. By analyzing recency, frequency, and monetary metrics, we identified distinct customer segments, uncovering valuable insights to guide targeted marketing strategies.

K-Means clustering effectively grouped customers, allowing businesses to prioritize high-value customers, re-engage at-risk groups, and tailor retention efforts. Additionally, the analysis of purchasing trends, seasonal patterns, and geographic distribution provided deeper insights into customer preferences and behavior.